

Bonnes Pratiques du Milieu Pro SNI

Code embarqué maintenable · Documentation technique · Sécurité & fiabilité
Normes industrielles · Travail en équipe multidisciplinaire

Ce que l'industrie attend au-delà du code

Dans les domaines embarqués, automatique et IoT, la qualité du code est cruciale mais insuffisante. Les systèmes embarqués fonctionnent souvent dans des **environnements critiques** (médical, aérospatial, automobile). La documentation, la traçabilité, la sécurité et les normes sont autant importantes que la maîtrise technique.

1. Écrire du code embarqué de qualité professionnelle

MISRA C / MISRA C++

Standard de codage sécurisé pour systèmes embarqués critiques. Utilisé dans auto (ISO 26262), médical (IEC 62304), avionique (DO-178C). Évite les comportements indéfinis du C.

→ Bannir `malloc()` en systèmes critiques. Toujours initialiser les variables. Pas de récursion.

Gestion des interruptions

Les ISR doivent être courtes et non-bloquantes. Utiliser des flags volatiles ou queues FreeRTOS. Jamais de `printf()` dans une ISR. Désactiver les interruptions le moins longtemps possible.

→ `volatile uint8_t flag_uart_rx = 0;` // dans ISR, traitement dans la tâche principale

Gestion de la mémoire statique

En embarqué critique, pas d'allocation dynamique (heap) à runtime. Toutes les structures sont allouées statiquement. Évite la fragmentation mémoire et les erreurs runtime.

→ `static Buffer rx_buffer[MAX_BUFFER_SIZE];` — alloué à la compilation

Watchdog timer

Toujours configurer le watchdog pour redémarrer le système si le programme se bloque. Le rafraîchir uniquement quand le système fonctionne correctement.

→ Un système qui redémarre proprement est meilleur qu'un système gelé en production.

Tests et validation embarquée

Unit tests sur PC d'abord (Unity Test Framework, CppUTest), puis validation sur cible. Hardware-in-the-loop (HIL) pour validation système complète.

→ *Tester la logique métier séparément du hardware avec des mocks/stubs.*

Versioning firmware

Versionner le firmware (Major.Minor.Patch) dans une constante ROM. Logger la version au démarrage. Permet le traçage en production.

→ `#define FIRMWARE_VERSION "2.3.1" // stocké en Flash ROM`

2. Documentation technique — le standard industriel

Fiche technique (Datasheet) analysis

- Savoir lire et interpréter une datasheet de composant électronique.
- Identifier les limites absolues, conditions de fonctionnement typiques, timing diagrams.
- Ne jamais supposer : tout est dans la datasheet.

Dossier de conception technique (DCT)

- Document décrivant l'architecture matérielle et logicielle du système.
- Schémas blocs, diagrammes de séquence, spécifications des interfaces.
- Standard dans les grands groupes (Airbus, Thales, Valeo) — document de référence.

Plan de test & Rapport de validation

- Définir les cas de test avant de coder (TDD : Test-Driven Development).
- Couvrir les cas nominaux ET les cas aux limites (valeurs min/max/hors-plage).
- Traçabilité exigences ↔ tests : chaque exigence doit être testée.

Rapport de métrologie / calibrage

- Documenter la procédure de calibrage de chaque capteur.
- Calcul d'incertitude de mesure selon GUM (Guide to Uncertainty in Measurement).
- Traçabilité métrologique : raccordement aux étalons nationaux (LNE en France).

Commentaires Doxygen dans le code

- Commenter toutes les fonctions publiques avec Doxygen (@brief, @param, @return).
- Générer automatiquement la documentation HTML avec doxygen -g.
- Standard dans les équipes pro : la doc fait partie du code, pas en option.

3. Normes industrielles à connaître

Ces normes sont mentionnées dans les offres d'emploi. Les connaître donne un avantage décisif :

Norme	Domaine	Ce qu'elle impose
ISO 26262	Automobile (ASIL)	Sécurité fonctionnelle des systèmes électroniques dans les véhicules. Niveaux ASIL A à D.
IEC 61508	Sécurité fonctionnelle générale	Standard mère de sécurité fonctionnelle. SIL 1 à 4. Base de l'ISO 26262 et IEC 62061.
DO-178C	Avionique (DAL)	Logiciel embarqué aéronautique. Niveaux DAL A à E. Extrêmement rigoureux.
IEC 62304	Dispositifs médicaux	Logiciel pour dispositifs médicaux. Classes A, B, C selon risque pour le patient.
IEC 62443	Cybersécurité industrielle	Sécurité des systèmes de contrôle industriels (ICS/SCADA). SL 1 à 4.

MISRA C:2012	Codage embarqué sûr	Règles de codage C pour éviter les comportements indéfinis. Requis en auto/médical.
IEC 61131-3	Automates industriels	Langages de programmation API : Ladder, ST, FBD, IL, SFC. Standard mondial API.
IEEE 1451	Instrumentation & capteurs	Standard pour capteurs intelligents et interfaces transducteur/réseau.

4. Travailler en équipe multidisciplinaire

Un ingénieur SNI travaille avec des électroniciens, des mécaniciens, des physiciens, des chefs de projet et des clients. Savoir communiquer avec chaque interlocuteur est une compétence autant demandée que les compétences techniques.

■ Avec les électroniciens

- Lire et comprendre les schémas KiCad/Altium.
- Donner des specs logicielles claires (temps de réponse, protocoles, niveaux de tension).
- Participer aux revues de schéma pour détecter les problèmes logiciel/matériel tôt.

■ Avec les mécaniciens

- Comprendre les contraintes d'encombrement et de températures.
- Anticiper les vibrations et chocs (IEC 60068 — tests environnementaux).
- Documenter les besoins en connecteurs et câblage.

■ Avec les physiciens / métrologues

- Comprendre les principes physiques des capteurs utilisés.
- Dialoguer sur les incertitudes et les limites de mesure.
- Valider ensemble les protocoles de calibrage.

■ Avec les chefs de projet

- Estimer honnêtement les délais — buffer de 30% pour les imprévus embarqués.
- Utiliser JIRA, GitLab Issues ou MS Project pour le suivi.
- Alerter tôt sur les risques techniques, ne pas attendre la dernière minute.